



Projet – Application Planning MLA / ICC

Description

Cet espace Confluence centralise l'ensemble de la documentation, des décisions et du suivi du projet **Application de Planning du Ministère de la Louange et de l'Adoration (MLA)** pour les campus ICC.

Il sert de point d'entrée unique pour les responsables métier, l'équipe projet et les développeurs afin de comprendre la vision, les règles métier, l'architecture et l'avancement du produit.

Démarrer ici

-  **Nouveau sur le projet ?** Commence par → *Vision & Contexte*
-  **Tu travailles sur les règles métier ?** → *Fonctionnel & Règles Métier*
-  **Tu développes ?** → *Architecture Technique*
-  **Tu pilotes le projet ?** → *Roadmap & Planning*

Project Tracker

Activité du projet

Ce bloc affiche automatiquement les dernières mises à jour de la documentation et permet de suivre l'avancement du travail en temps réel.





We can't show this project.

You don't have permission to view it, or it doesn't exist. Contact your admin to request access.



Accès rapide aux pages clés

Section	Lien
 Vision & Contexte	Ici
 Fonctionnel & Métier	
 Cas d'utilisation	
 Architecture	
 Roadmap	
 Guide utilisateur	

Indicateurs projet

Indicateurs clés

- Phase actuelle : **Conception & cadrage**
- Version cible : **MVP MLA v1.0**
- Campus pilote : **Toulouse**
- Responsable produit : **Amos DORCEUS**
- Dernière mise à jour : Jan 27, 2026

Recently updated content

 This list below will automatically update each time somebody in your space creates or updates content.

EP-06 - Moteur de Planning

Feb 13, 2026 • contributed by T-lamo

Épique : Gestion de la Polyvalence et des Référentiels Métiers (V2.1)

Feb 09, 2026 • contributed by T-lamo

EP-05 - Gestion des Collectifs & Indisponibilités

Feb 02, 2026 • contributed by T-lamo

Épique EP-04 - Référentiel RH & Chantres

Feb 02, 2026 • contributed by T-lamo

Épique EP-03 : Socle Organisationnel

Feb 02, 2026 • contributed by T-lamo

Épique 2 — Gestion des Activités & Planning MLA (API)

Feb 02, 2026 • contributed by T-lamo

Untitled whiteboard 2026-01-31

Jan 29, 2026 • contributed by T-lamo

Épique 1 — Authentification & Gestion des Utilisateurs (API)

Jan 29, 2026 • contributed by T-lamo

Epique 0

Jan 29, 2026 • contributed by T-lamo

Contributors

 This list below will automatically update each time somebody in your space creates or updates content.

[T-lamo](#)





Cahier des Charges

Projet : Application de Planning de Service pour la Chorale MLA – ICC

Auteur / Décideur / MOA / MOE : Amos DORCEUS

Version : 1.0

Date : Jan 28, 2026

- 📌 1. Présentation Générale
 - 1.1 Contexte
 - 1.2 Objectif du projet
- 🕒 2. Gouvernance du Projet
- 🚧 3. Périmètre
 - 3.1 Inclus
 - 3.2 Hors périmètre
- ⚙️ 4. Exigences Non Fonctionnelles
 - 4.1 Sécurité
 - 4.2 Performance
 - 4.3 Disponibilité
 - 4.4 Conformité
- 🔧 5. Règles de Gestion
- 🏗️ 6. Architecture Cible
 - 6.1 Schéma logique
 - 6.2 Séparation des couches
- 🌱 7. Modèle Métier
 - Entités principales
- 🖋️ 8. Cas d'Utilisation
 - UC1 – Authentification
 - UC2 – Gestion des utilisateurs
 - UC3 – Créer activité
 - UC4 – Consulter calendrier
 - UC5 – Constituer planning
 - UC6 – Affecter chantres
 - UC7 – Vérifier contraintes
 - UC8 – Valider planning
 - UC9 – Notifier chantres
 - UC10 – Gérer membres
 - UC11 – Gérer équipes
 - UC12 – Consulter profils
 - UC13 – Déclarer indisponibilité
 - UC14 – Consulter indisponibilités
 - UC15 – Statistiques
- 📅 9. Politique de Données
- 📅 10. Planning Prévisionnel
- 🔍 11. Traçabilité
- 🕒 12. Journal des Décisions
- ✅ 14. Validation

1. Présentation Générale

Objectif du document

Ce cahier des charges formalise les besoins fonctionnels, organisationnels et techniques pour le développement d'une application web de gestion du planning de service du **Ministère de la Louange et de l'Adoration (MLA)** au sein des églises **ICC**.

1.1 Contexte

ICC est une organisation ecclésiale internationale structurée hiérarchiquement :

- Église mère (campus mondial)
- Campus principaux par pays
- Campus par ville

Chaque campus fonctionne selon une organisation standardisée en ministères.

Le **MLA (Ministère de la Louange et de l'Adoration)** est l'un des ministères les plus sollicités, nécessitant une gestion rigoureuse des équipes, des plannings et des rôles.

Les outils actuels (Excel, fichiers partagés) ne répondent plus aux besoins de :

- Fiabilité
- Traçabilité
- Communication
- Scalabilité

1.2 Objectif du projet

Développer une **application web sécurisée et extensible** permettant :

- La gestion centralisée du planning du MLA
- La programmation des chantres
- La gestion des indisponibilités
- La communication automatisée

Cette application devra pouvoir évoluer vers une plateforme multi-ministères et multi-campus.

2. Gouvernance du Projet

Le projet est piloté par @T-lamo qui assure la totalité des décisions fonctionnelles et techniques.

Rôle	Responsable	Missions
Sponsor	@T-lamo	Vision, validation finale
MOA	@T-lamo	Expression du besoin
MOE	@T-lamo	Conception & développement
Chef de projet	@T-lamo	Planning, risques, arbitrage
Recette	@T-lamo	Validation fonctionnelle

3. Périmètre

3.1 Inclus

- Authentification et gestion des rôles
- Gestion des membres MLA
- Gestion des équipes
- Gestion des activités
- Programmation des chantres
- Gestion des indisponibilités
- Notifications
- Statistiques

3.2 Hors périmètre

- Gestion financière
- Paie / RH
- Dons et offrandes
- Streaming / multimédia
- Application mobile native

4. Exigences Non Fonctionnelles

4.1 Sécurité

- Authentification JWT
- Hashage des mots de passe (Argon2 / Bcrypt)
- Gestion des rôles RBAC
- Journal d'audit

4.2 Performance

- Temps de réponse < 2 secondes
- Support de 500 utilisateurs simultanés

4.3 Disponibilité

- Disponibilité cible : 99%
- Sauvegarde quotidienne

4.4 Conformité

- RGPD
- Export et suppression des données personnelles

5. Règles de Gestion

ID	Règle
RG-01	Un chantre indisponible ne peut pas être programmé
RG-02	Une activité doit contenir au moins un choriste et un musicien
RG-03	Chaque musicien doit avoir un instrument et un rôle
RG-04	Chaque choriste doit avoir une voix
RG-05	Une activité validée est verrouillée
RG-06	Un chantre ne peut pas être affecté deux fois à la même activité

6. Architecture Cible

6.1 Schéma logique

- Frontend : Vue

- Backend : FastAPI
- Base de données : PostgreSQL
- Authentification : JWT
- Déploiement : Docker / Cloud

6.2 Séparation des couches

- Présentation
- Métier
- Données

7. Modèle Métier

Entités principales

- OrganisationICC
- Pays
- Campus
- Ministère
- MLA
- Membre
- Chantre
- Choriste
- Musicien
- Équipe
- Activité
- PlanningService
- Affectation
- Indisponibilité

8. Cas d'Utilisation

UC1 – Authentification

Acteurs : Tous

UC2 – Gestion des utilisateurs

Acteur : Administrateur

UC3 – Créer activité

Acteur : Responsable MLA

UC4 – Consulter calendrier

Acteurs : Tous

UC5 – Constituer planning

Acteur : Responsable MLA

UC6 – Affecter chantres

Acteur : Responsable MLA

UC7 – Vérifier contraintes

Acteur : Système

UC8 – Valider planning

Acteur : Responsable MLA

UC9 – Notifier chantres

Acteur : Système

UC10 – Gérer membres

Acteur : Responsable MLA

UC11 – Gérer équipes

Acteur : Responsable MLA

UC12 – Consulter profils

Acteurs : Tous

UC13 – Déclarer indisponibilité

Acteur : Chantre

UC14 – Consulter indisponibilités

Acteurs : Responsable / Chantre

UC15 – Statistiques

9. Politique de Données

Donnée	Conservation
Activités	5 ans
Logs connexions	12 mois
Comptes inactifs	Désactivés après 24 mo

10. Planning Prévisionnel

Phase	Durée	Livrable
Cadrage	1 semaine	Cahier validé
Conception	2 semaines	UML / MLD
Développement	15 semaines	Version bêta
Recette	1 semaine	Validation
Déploiement	1 semaine	Version 1.0

11. Traçabilité

Besoin	UC	Règle
Authentification	UC1	RG-01
Programmation	UC5	RG-02
Indisponibilité	UC13	RG-01

12. Journal des Décisions

Date	Décision	Justification
Jan 14, 2026	MOA unique	Simplification gouvernance
Jan 20, 2026	JWT retenu	Sécurité moderne

-
- Diagrammes UML
 - Modèle de base de données
 - Maquettes UI
 - API Documentation

✓ 14. Validation

Nom	Rôle	Date	Signature
MOA Unique @T-lamo	Décideur	Jan 28, 2026	✓

Sprint 0 — Mise en Place du Projet MLA / ICC

Projet : Application de Planning de Service – MLA / ICC

Sprint : 0 (Initialisation)

Durée : 1 à 2 semaines

Responsable : @T-lamo → Amos DORCEUS

Objectif : Mettre en place les fondations fonctionnelles, techniques et organisationnelles du projet avant le développement des fonctionnalités métier.

1. Objectifs du Sprint 0

Le Sprint 0 a pour but de :

- Poser le cadre du projet
- Sécuriser la vision produit
- Mettre en place l'environnement technique
- Préparer le backlog et les outils de suivi
- **Concevoir les diagrammes UML et le modèle de base de données**

À l'issue du Sprint 0, le projet doit être **prêt à démarrer le Sprint 1 de développement**.

2. Périmètre du Sprint 0

Inclus

- Validation du cahier des charges
- Définition de l'architecture technique
- Mise en place des environnements (dev, test)
- Initialisation du dépôt de code
- Mise en place de la base de données
- **Conception des diagrammes UML : Use Cases, Classes, Activités**
- **Création du MLD et schéma relationnel**
- Définition du backlog produit
- Définition des standards de développement

Hors périmètre

- Développement de fonctionnalités métier

- Déploiement en production
- Intégration des notifications

3. Gouvernance du Sprint

Rôle	Responsable	Missions
Product Owner	@T-lamo	Priorisation du backlog
Scrum Master	@T-lamo	Animation du sprint
Développeur	@T-lamo	Implémentation technique
Testeur	@T-lamo	Vérification livrables

4. Livrables Attendus

ID	Livrable	Description
L-01	Cahier des charges validé	Version figée pour dev
L-02	Architecture technique	Schéma et stack validés
L-03	Dépôt Git	Repo initialisé
L-04	Environnement dev	Backend + Front configurés
L-05	Schéma base de données	MLD / UML / migrations
L-06	Diagrammes UML	Use Cases, Classes, Activités
L-07	Backlog produit	User stories priorisées
L-08	Plan de tests	Stratégie de validation

5. Architecture Technique Cible

Stack

- Frontend : Vue
- Backend : FastAPI
- Base de données : PostgreSQL

- Authentification : JWT
- ORM : SQLModel
- CI/CD : GitHub Actions
- Déploiement : Docker

Environnements

- Développement local
- Environnement de test

8. Critères de Réussite (Definition of Done – Sprint 0)

Le Sprint 0 est terminé lorsque :

- Cahier des charges validé
- Dépôt Git opérationnel
- Backend et DB fonctionnels
- JWT fonctionnel en dev
- Backlog Sprint 1 priorisé
- Diagrammes UML disponibles et validés
- MLD et schéma relationnel finalisés

9. Planning Prévisionnel

Jour	Activité
J1	Validation du cahier des charges
J2	Setup Git + Environnements
J3	Setup Backend + DB
J4	JWT + ORM + Diagrammes UML
J5	MLD / Schéma relationnel
J6	Backlog + Revue Sprint 0

10. Risques Identifiés

ID	Risque	Impact	Mitigation
R-01	Mauvaise configuration DB	Fort	Documentation + tests
R-02	JWT mal implémenté	Fort	Revue sécurité
R-03	Périmètre flou	Moyen	Validation écrite
R-04	Diagrammes incomplets	Moyen	Validation par MOA

10. Risques Identifiés

ID	Risque	Impact	Mitigation
R-01	Mauvaise configuration DB	Fort	Documentation + tests
R-02	JWT mal implémenté	Fort	Revue sécurité
R-03	Périmètre flou	Moyen	Validation écrite
R-04	Diagrammes incomplets	Moyen	Validation par MOA

11. Rituels Agile

- Sprint Planning : Début Sprint
- Daily (auto) : Suivi journalier
- Sprint Review : Fin Sprint
- Rétrospective : Amélioration continue

12. Journal des Décisions — Sprint 0

Date	Décision	Justification
Jan 7, 2026	Stack FastAPI + PostgreSQL	Performance et maintenabilité
Jan 7, 2026	JWT retenu	Sécurité moderne
Jan 7, 2026	Docker utilisé	Portabilité environnement

Jan 7, 2026	UML et MLD obligatoires	Préparer le développement Sprint 1
-------------	-------------------------	---------------------------------------

Épique 0

Épique 0 — Initialisation du Projet

ID	User Story	Priorité	Statut
US-00.1	En tant que MOA, je veux valider le cahier des charges afin de figer le périmètre	Haute	À faire
US-00.2	En tant que développeur, je veux créer le dépôt Git afin de versionner le code	Haute	À faire
US-00.3	En tant que développeur, je veux configurer FastAPI afin de démarrer l'API	Haute	À faire
US-00.4	En tant que développeur, je veux configurer la base PostgreSQL afin de stocker les données	Haute	À faire
US-00.5	En tant que développeur, je veux configurer JWT afin de	Haute	À faire

	sécuriser l'authentification		
US-00.6	En tant que développeur, je veux générer les migrations DB afin de versionner le schéma	Haute	À faire
US-00.7	En tant que MOA, je veux définir le backlog Sprint 1 afin de démarrer le développement métier	Haute	À faire
US-00.8	En tant que MOA, je veux créer tous les diagrammes UML pour avoir une vision claire de l'architecture	Haute	À faire

Épique 1 — Authentification & Gestion des Utilisateurs (API)

Backlog — User Stories

ID	User Story	Priorité	Statut
US-01.1	En tant qu'utilisateur, je veux me connecter avec mes identifiants afin d'accéder à l'application	Haute	À faire
US-01.2	En tant qu'utilisateur, je veux me déconnecter afin de sécuriser mon compte	Haute	À faire
US-01.3	En tant qu'administrateur ICC, je veux créer un compte utilisateur afin de donner accès à l'application	Haute	À faire
US-01.4	En tant qu'administrateur ICC, je veux modifier un compte utilisateur afin de maintenir les informations à jour	Moyenne	À faire

US-01.5	En tant qu'administrateur ICC, je veux activer ou désactiver un compte afin de contrôler les accès	Haute	À faire
US-01.6	En tant qu'administrateur ICC, je veux attribuer un ou plusieurs rôles à un utilisateur afin de définir ses permissions	Haute	À faire
US-01.7	En tant que système, je veux vérifier les droits d'accès afin de sécuriser les endpoints de l'API	Haute	À faire
US-01.8	En tant qu'utilisateur, je veux voir mon profil afin de consulter mes informations personnelles	Moyenne	À faire
US-01.9	En tant qu'utilisateur, je veux modifier mon mot de passe afin de	Haute	À faire

	sécuriser mon compte		
US-01.10	En tant qu'administrateur ICC, je veux consulter la liste des utilisateurs afin de gérer les accès	Moyenne	À faire

Dépendances

- Épique 0 — Initialisation du Projet terminé
- Base de données et JWT configurés
- Modèle Utilisateur / Rôle disponible

Critères de complétion de l'Épique

L'épique est considéré comme terminé lorsque :

- Tous les endpoints d'authentification sont fonctionnels
- Les rôles sont appliqués aux accès API
- Les comptes utilisateurs sont gérables par l'administrateur
- La sécurité JWT est validée par des tests

Indicateurs de succès

- 100% des endpoints sensibles protégés par JWT
- Création et connexion utilisateur opérationnelles
- Gestion des rôles fonctionnelle
- Aucun accès non autorisé possible

Épique 2 — Gestion des Activités & Planning MLA (API)

Objectif

Mettre en place la **gestion des activités** du Ministère de la Louange et de l'Adoration (MLA) et permettre aux responsables de **constituer le planning de service des chantres** via l'API.

Tout se fait côté **back-end**, sans interface utilisateur. L'API devra gérer : création, modification, consultation des activités, affectation des chantres et vérification des contraintes.

PARTIE 1 : Structure Stratégique (Vue Confluence)

Le projet est divisé en **4 Épiques** majeures, classées par ordre de dépendance technique et priorité métier.

Épique	Nom	Objectif Métier	Modèles concernés	Priorité
EP-01	Socle Organisationnel	Définir la structure géographique et administrative (Campus, Ministères).	OrganisationICC, Pays, Campus, Ministère, Pole	P0 (Bloquant)
EP-02	Référentiel RH & Chantres	Gérer l'annuaire des membres et leurs spécificités techniques	Membre, Chantre, Choriste, Musicien, Voix,	P1 (Haute)

		(Voix/Instruments).	Instrument	
EP-03	Gestion des Collectifs	Organiser les membres en équipes et gérer leurs indisponibilités.	Equipe, Equipe_Membre, Indisponibilite, Responsabilite	P2 (Moyenne)
EP-04	Moteur de Planning	Planifier les activités et affecter les ressources selon les contraintes.	Activite, PlanningService, StatutPlanning, Affectation	P1 (Critique)

 PARTIE 2 : Backlog Opérationnel (Tickets Jira)

Épique 01 : Socle Organisationnel

Ticket ID	Titre	Description (User Story)	Critères d'Acceptation (AC)	Qualité & Tests (pytest)
MLA-101	CRUD Référentiel Geo	En tant qu'admin, je veux configurer les pays et organisations.	Endpoints: POST/GET sur /pays et /organisations.	Test de création avec organisation_id inexistant (doit échouer).

			Validation des UUID.	
MLA-102	Gestion Campus & Pôles	En tant qu'admin, je veux créer les campus et les pôles associés.	Endpoints: <code>/campus</code> et <code>/poles</code> . Suppression en cascade fonctionnelle.	Vérifier que la suppression d'un Campus supprime les Ministères associés.

Épique 02 : Référentiel RH & Chantres

Ticket ID	Titre	Description (User Story)	Critères d'Acceptation (AC)	Qualité & Tests (pytest)
MLA-201	Enrôlement Membre	En tant que resp., je veux créer un membre rattaché à un pôle.	<code>POST /membres</code> . Obligation d'avoir <code>ministere_id</code> et <code>pole_id</code> .	Test d'intégrité : un membre ne peut exister sans ministère.
MLA-202	Profil Technique Chantre	En tant que chantre, je veux définir ma voix ou mon instrument.	Endpoints: <code>/chantres</code> , <code>/choristes</code> . Support multi-rôles (Chantre + Musicien).	Vérifier qu'un <code>Choriste</code> est lié à un code de <code>Voix</code> valide (Enum).

Épique 03 : Gestion des Collectifs

Ticket ID	Titre	Description (User Story)	Critères d'Acceptation (AC)	Qualité & Tests (pytest)
MLA-301	Gestion des Équipes	En tant que leader, je veux constituer des équipes de service.	CRUD sur <code>/equipes</code> et table de liaison <code>/equipes/{id}/membres</code> .	Test de doublon : un membre ne peut pas être ajouté deux fois à la même équipe.
MLA-302	Registre d'Indisponibilité	En tant que chantre, je veux déclarer mes absences.	POST <code>/indisponibilite</code> S. Format date ISO 8601.	Vérifier que la date de fin est postérieure à la date de début.

Épique 04 : Moteur de Planning (Objectif Final)

Ticket ID	Titre	Description (User Story)	Critères d'Acceptation (AC)	Qualité & Tests (pytest)
MLA-401	Création d'Activité	En tant que resp. MLA, je veux créer un culte ou une répétition.	POST <code>/activites</code> . Liaison avec <code>Campus</code> .	Vérifier que l'activité est créée avec le bon fuseau horaire.
MLA-402	Affectation & Conflits	En tant que planificateur, je veux affecter un	POST <code>/affectations</code> . Vérification automatique	Test Critique : Échec de l'affectation si le chantre a une

		chantre à un planning.	du statut_code .	Indisponibilité sur ce créneau.
--	--	------------------------	------------------	---------------------------------

Backlog — User Stories

ID	User Story	Priorité	Statut
US-02.1	En tant que responsable MLA, je veux créer une activité via l'API afin de planifier un culte, répétition ou APP	Haute	À faire
US-02.2	En tant que responsable MLA, je veux modifier une activité via l'API afin de mettre à jour ses informations	Moyenne	À faire
US-02.3	En tant que responsable MLA, je veux consulter la liste des activités via l'API afin d'obtenir les informations nécessaires pour le planning	Haute	À faire

US-02.4	En tant que responsable MLA, je veux sélectionner une équipe via l'API pour une activité afin de préparer les affectations	Haute	À faire
US-02.5	En tant que responsable MLA, je veux affecter les chantres à une activité via l'API en précisant leur rôle (voix/instrument) afin de constituer le planning	Haute	À faire
US-02.6	En tant que système (API), je veux vérifier automatiquement les contraintes (indisponibilités, voix, instruments, rôle) afin de valider la planification	Haute	À faire
US-02.7	En tant que responsable MLA, je veux valider le planning via l'API	Haute	À faire

	afin que toutes les affectations soient confirmées et verrouillées côté back-end		
US-02.8	En tant que responsable MLA, je veux consulter les indisponibilités des chantres via l'API afin de planifier correctement	Moyenne	À faire
US-02.9	En tant que responsable MLA, je veux récupérer des statistiques sur les activités et les affectations via l'API afin de suivre la planification	Moyenne	À faire

ID	User Story	Priorité
US-M.1	En tant que Responsable, je veux créer un membre et l'associer à un ministère/pôle pour l'intégrer au système.	Haute

US-M.2	En tant que Responsable, je veux définir un membre comme Chantre (Choriste ou Musicien) avec ses spécificités (voix/instrument).	Haute
US-M.3	En tant que Responsable, je veux créer une équipe et y ajouter des membres pour faciliter la planification groupée.	Haute
US-M.4	En tant que Chantre, je veux déclarer mes indisponibilités via l'API pour ne pas être sollicité inutilement.	Moyenne

Dépendances

- Épique 1 — Authentification & Gestion des Utilisateurs terminé
- Épique 2 (Gestion des membres et équipes) opérationnel côté API
- JWT fonctionnel pour sécuriser les endpoints API

Critères de complétion de l'Épique

L'épique est considéré comme terminé lorsque :

- Tous les endpoints API de création, modification et consultation des activités sont fonctionnels
- Les affectations respectent les contraintes métiers (voix, instruments, indisponibilités)
- Les plannings peuvent être validés côté back-end
- Les statistiques sont récupérables via l'API

Indicateurs de succès

- Création/modification/consultation des activités opérationnelle via API
 - Affectations correctes et contraintes vérifiées côté API
 - Validation du planning côté back-end
 - Statistiques disponibles pour le responsable MLA via API
-

Épique EP-03 : Socle Organisationnel

Objectif Métier

Structurer l'écosystème ICC pour permettre une gestion multi-sites (Campus) et multi-départements (Ministères/Pôles). Cela garantit que chaque activité et chaque membre est correctement localisé et rattaché à une autorité administrative.

Règles Métiers (Business Rules)

1. **Hiérarchie Stricte** : Un Pôle appartient obligatoirement à un Ministère, qui appartient à un Campus, qui appartient à un Pays, qui appartient à une Organisation.
2. **Unicité** : Deux campus ne peuvent pas avoir le même nom au sein d'un même Pays.
3. **Intégrité de Suppression** : La suppression d'un Campus entraîne la suppression en cascade de ses Ministères, Pôles et Activités associées (Contrainte **CASCADE**).
4. **Sécurité** : Seuls les utilisateurs ayant le rôle **ADMIN_GLOBAL** ou **ADMIN_PAYS** peuvent modifier cette structure.

User Stories (Le "Quoi")

ID	User Story	Priorité
US-01.1	En tant qu'Administrateur, je veux créer une Organisation et ses Pays afin de définir le périmètre de l'église.	Haute
US-01.2	En tant qu'Administrateur, je veux créer un Campus rattaché à un pays pour localiser les activités.	Haute
US-01.3	En tant que Responsable, je veux configurer les Ministères (ex: MLA)	Haute

	au sein de mon campus.	
US-01.4	En tant que Responsable, je veux créer des Pôles (ex: Pôle Chantre) au sein de mon ministère.	Haute

Tickets JIRA (Le "Comment")

Ticket MLA-101 : Infrastructure Géo (Organisation & Pays)

- **Description** : Création des endpoints CRUD pour `t_organisationicc` et `t_pays`.
- **Critères d'Acceptation** :
 - Endpoint `POST /organisations` et `POST /pays`.
 - Validation : un Pays doit être lié à une Organisation existante.
- **Qualité & Tests** : Test de création d'un pays avec un `organisation_id` invalide (doit retourner 400/404).

Ticket MLA-102 : Gestion des Campus

- **Description** : CRUD pour `t_campus` avec liaison vers `t_pays`.
- **Critères d'Acceptation** :
 - Endpoints `GET`, `POST`, `PUT`, `DELETE` sur `/campus`.
 - Le JSON de retour doit inclure le nom du pays parent.
- **Qualité & Tests** : Test de suppression d'un pays : vérifier que le campus associé est bien supprimé (Cascade).

Ticket MLA-103 : Gestion des Ministères

- **Description** : CRUD pour `t_ministere` rattaché à `t_campus`.
- **Critères d'Acceptation** :
 - Endpoint `POST /ministeres`.
 - L'ID du campus doit être validé en base avant insertion.
- **Qualité & Tests** : Test unitaire vérifiant que l'on peut lister tous les ministères d'un campus spécifique via `/campus/{id}/ministeres`.

- **Description :** CRUD pour `t_pole` rattaché à `t_ministere`.
- **Critères d'Acceptation :**
 - Endpoint `POST /poles`.
 - Validation : un pôle ne peut pas être créé "orphelin" sans ministère.
- **Qualité & Tests :** Test d'intégration : Créer un Campus → Un Ministère → Un Pôle, puis vérifier la chaîne de relations en base de données.

Épique EP-04 - Référentiel RH & Chantres

 Le Master Prompt à copier

Objet : Conception détaillée de l'Épique EP-02 - Référentiel RH & Chantres

Rôle : Agis en tant que Product Owner et Lead Développeur expert en écosystèmes FastAPI / SQLAlchemy.

Contexte : > Nous développons l'API "MLA" pour la gestion du planning d'un ministère de louange. L'Épique 01 (Socle Organisationnel) est terminée. Nous devons maintenant implémenter l'**Épique 02 : Référentiel RH & Chantres**.

Données sources (Modèles SQLAlchemy concernés) :

- **Membre** (lié à Ministère/Pôle)
- **Chantre** (lié à Membre)
- **Choriste** (lié à Chantre et Voix)
- **Musicien** (lié à Chantre et Instrument)
- Tables de référence : **Voix**, **Instrument**

Livrables attendus :

1. **Documentation Confluence (Niveau Stratégique) :**

- **Objectif de l'Épique :** Définir la valeur métier de la centralisation des compétences.
- **Règles Métiers (Business Rules) :** Liste exhaustive (ex: un Chantre peut être à la fois Choriste et Musicien, l'unicité du membre, la gestion des voix par défaut).
- **Diagramme de flux logique :** Expliquer le passage de "Membre" à "Chantre spécialisé".

2. **Backlog JIRA (Niveau Opérationnel) :** Génère des tickets JIRA détaillés.

Pour chaque ticket, inclus :

- **Titre et User Story :** Format "En tant que [Rôle], je veux [Action] afin de [Bénéfice]".
- **Critères d'Acceptation (AC) :** Liste technique des endpoints (ex: **POST** **/chantres**), validations de schémas Pydantic, et codes de retour HTTP

(201, 400, 404).

- **Stratégie de Test (Pytest)** : Détaille les tests unitaires et d'intégration nécessaires (ex: tester qu'un Choriste ne peut pas être créé sans une Voix valide, tester la relation One-to-One entre Utilisateur et Membre).

Contraintes techniques à respecter :

- Utilisation de **UUID v4** pour tous les IDs.
- Gestion rigoureuse des **clés étrangères** (un membre doit appartenir à un `pole_id` et `ministere_id` existants).
- Séparation des schémas Pydantic : `Base` , `Create` , `Update` , et `Read` (pour éviter les récursions infinies dans les relations).

Objectif de l'Épique

Centraliser les informations administratives et les compétences techniques (vocales et instrumentales) de chaque membre du ministère. L'objectif est de permettre au moteur de planning (EP-04) de filtrer intelligemment les ressources selon les besoins d'un service (ex: "Besoin d'un Bassiste qui est aussi Soprano").

Règles Métiers (Business Rules)

- **Identité Unique** : Un individu ne peut avoir qu'un seul profil `Membre` . Un `Membre` peut être lié à un seul `Utilisateur` (Relation 1:1).
 - **Spécialisation Cumulative** : Un `Chantre` peut être simultanément `Choriste` (avec une voix principale/secondaire) et `Musicien` (avec un ou plusieurs instruments).
 - **Intégrité Structurale** : Un membre peut être rattaché à un `Ministere` et un `Pole` existants.
 - **Référentiel Strict** : Les voix et instruments doivent correspondre aux codes définis dans les tables de référence (`t_voix` , `t_instrument`).
 - **Délégation de Responsabilité** : Seul un responsable ayant les permissions adéquates peut modifier le profil technique (Choriste/Musicien) d'un membre.
-

📋 2. Backlog JIRA (Vue Opérationnelle)

Groupe 1 : Données de Référence (Préalable)

Ticket ID	Titre & User Story	Critères d'Acceptation (AC)	Stratégie de Test (Pytest)
MLA-201	Initialisation Voix & Instruments En tant que système, je veux disposer d'un référentiel de voix et d'instruments.	Endpoints: GET <code>/ref/voix</code> , GET <code>/ref/instruments</code> . Peupler <code>t_voix</code> (SOPRANO, ALTO, etc.) via un script de seed.	Tester que l'insertion d'un code de voix non présent dans l'Enum <code>VoixEnum</code> échoue.

Groupe 2 : Gestion des Membres (Administratif)

Ticket ID	Titre & User Story	Critères d'Acceptation (AC)	Stratégie de Test (Pytest)
MLA-202	CRUD Membres En tant que Responsable, je veux créer et gérer les membres du ministère.	POST <code>/membres</code> : Validation UUID pour <code>pole_id</code> et <code>ministere_id</code> .	Test d'intégration : Créer un membre et vérifier la 404 si le <code>pole_id</code> est aléatoire. Test Cascade : Supprimer un

		Schéma MembreRea d excluant les mots de passe.	membre doit supprimer son profil Chantre .
MLA-203	Liaison Utilisateur- Membre En tant qu'admin, je veux lier un compte utilisateur à une fiche membre.	Endpoint PATCH /utilisateur urs/{id}/l ink- membre . Vérifier l'unicité (un membre ne peut pas avoir deux utilisateurs).	Tester la contrainte UniqueCons traint sur membre_id dans la table t_utilisat eur .

Groupe 3 : Spécialisation technique (Métier MLA)

Ticket ID	Titre & User Story	Critères d'Acceptation (AC)	Stratégie de Test (Pytest)
MLA-204	Profil Chantre & Choriste En tant que Responsable, je veux définir la voix d'un membre.	POST /chantres (crée l'entrée pivot). POST /choriste S : liaison avec voix_code .	Test Unitaire : Vérifier que secondaire Voix est optionnel. Test de lecture : GET /membres/{ id} doit

			retourner les détails choriste si existants.
MLA-205	Profil Musicien En tant que Responsable, je veux définir les instruments pratiqués par un membre.	POST /musicien s . Validation du instrument _code contre la table t_instrument .	Tester l'ajout de plusieurs instruments pour un même chantre_id .

Contraintes techniques d'implémentation (Lead Dev)

1. **Schémas Circulaires** : Utilise des "Forward References" (**"Chantre"**) dans tes fichiers de schémas Pydantic pour que **MembreRead** puisse inclure **ChantreRead** sans erreur d'importation.
2. **DTO (Data Transfer Objects)** : * **MembreCreate** : Ne demande que les IDs et infos de base.
 - **MembreRead** : Utilise **lazy="joined"** dans SQLAlchemy pour récupérer le profil chantre en une seule requête SQL.
3. **Sécurité** : Tous les **POST/PATCH/DELETE** de cette épique doivent être protégés par une dépendance vérifiant la permission **SCOPE_RH_MANAGE** .

Épique : Gestion de la Polyvalence et des Référentiels Métiers (V2.1)

🎯 Objectif de l'Épique

Migrer d'une gestion de profil "statique" vers un système de **compétences dynamiques**. L'enjeu est de permettre une affectation intelligente (EP-04) basée sur des critères précis : le rôle (ex: "Basse"), la catégorie (ex: "Instrumentiste") et le niveau d'expertise, tout en respectant la nouvelle structure campus/ministère.

📋 Règles Métiers (Business Rules)

- Multi-spécialisation** : Un membre peut cumuler n'importe quel nombre de rôles (Chantre, Musicien, Technicien) sans limitation technique.
- Priorisation (Primary Role)** : Chaque membre doit avoir un et un seul rôle marqué `is_principal=True` . Ce rôle sera affiché par défaut sur son badge/profil.
- Référentiel Typé** : Un rôle (ex: "Alto") doit obligatoirement être rattaché à une catégorie (ex: "Chantre") pour faciliter les filtres de recherche.
- Gradation des Compétences** : Le niveau (DEBUTANT à EXPERT) est obligatoire pour chaque rôle associé afin d'éviter d'affecter un débutant sur un événement à fort enjeu.

📅 Backlog JIRA (Focus Polyvalence)

Groupe 1 : Référentiels et Hiérarchies

Ticket ID	Titre & User Story	Critères d'Acceptation (AC)	Stratégie de Test
POL-101	Catalogue des Catégories & Rôles	En tant qu'admin, je veux gérer les types de rôles disponibles dans le ministère.	Endpoints CRUD pour <code>t_categorie</code> et <code>t_rolecompetence</code> .
POL-102	Peuplement Initial (Seed)	En tant que système, je veux injecter les données de référence	Script de migration peuplant <code>t_categorie</code>

		vocales et instrumentales.	erole (VOCAL, INSTRUMENTAL, TECH) et les rôles associés.
--	--	----------------------------	--

Groupe 2 : Profil Technique du Membre

Ticket ID	Titre & User Story	Critères d'Acceptation (AC)	Stratégie de Test
POL-201	Gestion de la Polyvalence (Add/Remove)	En tant que Responsable, je veux ajouter ou retirer des compétences à un membre.	Endpoints POST/DELETE sur <code>/membres/{id}/roles</code> . Validation du <code>role_code</code> contre le référentiel.
POL-202	Définition du Rôle Principal	En tant que membre/RH, je veux définir quelle est ma fonction principale.	Endpoint PATCH pour basculer <code>is_principal</code> .
POL-203	Évaluation du Niveau	En tant que Coach, je veux mettre à jour le niveau d'un membre sur un rôle précis.	Endpoint PATCH sur la table de liaison <code>t_membre_role</code> pour modifier le champ <code>niveau</code> .

Détails techniques d'implémentation (Lead Dev)

1. Gestion du "Rôle Principal"

Pour garantir qu'un membre n'a qu'un seul rôle principal, deux approches sont possibles au niveau du code :

- **Niveau DB :** Utiliser un `Index` partiel unique (si supporté par la DB) sur `(membre_id)` où `is_principal = true`.
- **Niveau Application (Python) :** Dans le service de mise à jour, encapsuler dans une transaction :
 - a. `UPDATE t_membre_role SET is_principal = False WHERE membre_id = :id`
 - b. `UPDATE t_membre_role SET is_principal = True WHERE membre_id = :id AND role_code = :code`

EP-05 - Gestion des Collectifs & Indisponibilités

Objet : Planification détaillée de l'Épique EP-03 - Gestion des Collectifs & Indisponibilités

Rôle : Agis en tant que Product Owner et Architecte Solution Senior spécialisé en systèmes de planification (SaaS/ERP).

Contexte : > Nous développons l'API "MLA". Les fondations (Épique 01) et le référentiel des membres (Épique 02) sont opérationnels. Nous devons maintenant implémenter l'**Épique 03 : Gestion des Collectifs**. Cette étape est le "filtre" indispensable avant la génération des plannings.

Modèles de données à exploiter :

- **Equipe** : Groupement logique de membres.
- **Equipe_Membre** : Table associative (Many-to-Many).
- **Indisponibilite** : Créneaux où un Chantre est indisponible.
- **Responsabilite** : Rôles spécifiques au sein d'un Ministère/Pôle/Activité.

Livrables attendus :

1. **Documentation Confluence (Vision Métier) :**

- **Règles Métiers (Logic)** : Définis les contraintes (ex: Un membre peut-il appartenir à plusieurs équipes ?(Oui). Comment les responsabilités s'articulent-elles entre le Ministère et l'Équipe ?)
- **Gestion des Conflits** : Explique comment l'API doit traiter une affectation si une indisponibilité est enregistrée.

2. **Backlog JIRA (Découpage Technique)** : Génère des tickets JIRA structurés incluant :

- **Titre et User Story** : Format "En tant que [Persona], je veux [Fonction] afin de [Bénéfice]".
- **Critères d'Acceptation (AC)** : Spécifications des endpoints (ex: **POST** **/equipes/{id}/membres**), gestion des codes HTTP, et validations de données (ex: date de fin > date de début).

- **Qualité & Tests (Pytest)** : Scénarios de tests indispensables (ex: tester l'impossibilité d'ajouter un membre inexistant à une équipe, tester le chevauchement de deux périodes d'indisponibilité).

Contraintes d'implémentation :

- Sécurité : Les routes doivent vérifier les permissions via `t_role_permission` (ex: Seul un Responsable de Pôle peut modifier ses équipes).
- Performances : Pour la lecture des équipes, suggère l'utilisation de `selectinload` ou `joinedload` pour optimiser les jointures SQLAlchemy.
- Format : Utilise des tableaux pour les tickets JIRA et des sections claires pour Confluence.

Voici la planification détaillée de l'**Épique EP-03**. C'est une phase charnière : elle permet de passer d'une liste de membres individuels à une force opérationnelle organisée et qualifiée.

1. Documentation Confluence : Gestion des Collectifs & Indisponibilités

Vision Métier

L'objectif est de structurer les ressources humaines en unités de service (Équipes) et de sécuriser le processus de planification en intégrant les contraintes de présence (Indisponibilités) et les niveaux d'autorité (Responsabilités).

Règles Métiers (Logic)

- **Flexibilité des Équipes** : Un membre peut appartenir à plusieurs équipes (ex: Équipe Culte Dimanche et Équipe Répétition). La relation est gérée via la table associative `t_equipe_membre`.
- **Hiérarchie des Responsabilités** : La table `Responsabilite` définit le périmètre d'action d'un membre. Une responsabilité peut être globale au **Ministère**, spécifique à un **Pôle**, ou limitée à une **Activité** précise.

- **Priorité des Indisponibilités** : Une indisponibilité est absolue. Si un chanteur déclare une indisponibilité, il devient "non-éligible" pour toute affectation sur cette période.

Gestion des Conflits

Lors d'une tentative d'affectation (EP-04), l'API devra :

1. Vérifier l'intersection entre les dates de l'activité et les enregistrements de `t_indisponibilite` du chanteur.
2. Si un conflit existe, bloquer l'affectation avec un message d'erreur `409 Conflict`.
3. Permettre la consultation préventive des indisponibilités pour les planificateurs.

📋 2. Backlog JIRA (Découpage Technique)

Épique EP-03 : Tableaux des Tickets

Ticket ID	Titre & User Story	Critères d'Acceptation (AC)	Qualité & Tests (Pytest)
MLA-301	Moteur d'Équipes (CRUD) En tant que Responsable, je veux créer des équipes au sein de mon ministère.	- Endpoints <code>GET/POST /PUT/DELETE</code> sur <code>/equipes</code>. <code>ministere_id</code> obligatoire et valide. - Gestion du booléen <code>active</code>.	- Tester qu'une équipe ne peut être créée sans <code>ministere_id</code>. - Tester la suppression cascade : supprimer l'équipe doit vider <code>t_equipe_membre</code>.
MLA-302	Composition des Équipes	Endpoint <code>POST</code>	Tester l'ajout d'un membre

	En tant que Leader, je veux ajouter/retirer des membres à une équipe.	<code>/equipes/{id}/membres</code> (prend une liste d'IDs). Endpoint <code>DELETE /equipes/{id}/membres/{member_id}</code>.	inexistant (404). Vérifier qu'on ne peut pas ajouter deux fois le même membre à la même équipe.
MLA-303	Registre des Indisponibilités En tant que Chantre, je veux déclarer mes périodes d'absence.	- Endpoint <code>POST /indisponibilites</code>. - Validation : <code>date_fin > date_debut</code>. - Liaison obligatoire avec <code>chantre_id</code>.	Test critique : Bloquer l'enregistrement si les dates se chevauchent avec une indisponibilité existante du même chantre.

MLA-304	Attribution des Responsabilités En tant qu'Admin, je veux désigner les responsables de pôle/activité.	• CRUD sur <code>/responsabilites</code>. • Validation contextuelle (si <code>pole_id</code> est fourni, il doit exister). • Support des dates de mandat.	• Tester que la responsabilité est marquée <code>actif=False</code> automatiquement si la <code>dateFin</code> est dépassée.
----------------	---	---	--

Contraintes d'implémentation (Lead Dev)

Performances & Requête

Pour éviter le problème du "N+1 query", les lectures d'équipes doivent utiliser les chargements liés de SQLAlchemy :

```

1 Python
2
3 # Exemple de récupération optimisée
4 statement = select(Equipe).where(Equipe.id ==
5     equipe_id).options(selectinload(Equipe.membres_assoc))

```

Sécurité (RBAC)

L'accès aux endpoints doit être filtré :

- `GET /indisponibilites` : Le membre peut voir les siennes ; le responsable peut voir celles de son pôle.
- `POST /equipes` : Restriction stricte aux utilisateurs ayant la permission `MANAGE_TEAMS` dans le contexte du ministère.

EP-06 - Moteur de Planning

Objet : Planification de l'Épique EP-04 - Moteur de Planning (Cœur Critique)

Rôle : Agis en tant que Lead Developer et Product Owner expert en systèmes ERP et algorithmes de planification de ressources.

T'a proposition doit generer des ticket crud pour bien prendre en compte la gestion du planning. En tant que expert en python, po, lead dev ton role est d'indique les ticket à creer, les routes interessant pour bien gerer le planning en prenant en compte les tables qu'il faut. s'achant que le crud sur activité n'est pas encore réalisé. les table consernee son activite, statut planing, planningservice, slot, affectation, membre role, statut affectation, Indiponibilite (crud déjà realiser).

Contexte : > Nous arrivons à la phase finale et critique de l'API "MLA". Les fondations (EP-01), les ressources RH (EP-02) et les contraintes/collectifs (EP-03) sont prêts. Nous devons maintenant coder le **Moteur de Planning (EP-04)** qui orchestre tout.

Données sources (Modèles SQLModel) :

- **Activite** : L'événement physique (Culte, Répétition).
- **PlanningService** : L'instance de planning rattachée à une activité avec un **StatutPlanning**.
- **Affectation** : Le lien avec Role compétence
- Slot: different crenau d'un activité

Livrables attendus :

1. Documentation Confluence (Stratégie de Planning) :

- **Flux de Vie du Planning** : Définis les transitions de **StatutPlanning** (ex: Brouillon → Publié → Clôturé).
- **Logique de Validation Critique** : Explique comment le moteur doit interdire une affectation si :
 - a. Le membre a une **Indisponibilite** sur la date de l'activité.
 - b. Le Chantre est déjà affecté à un autre planning sur le même créneau.
- **Gestion des Rôles** : Comment l'affectation utilise les données de

RoleCompetence

2. **Backlog JIRA (Tickets Techniques)** : Génère des tickets détaillés incluant :

- **Titre & User Story** : Format "En tant que Planificateur, je veux [Action] afin de [Bénéfice]".
- **Volet Qualité & Tests (Pytest)** : Scénarios de "Stress Test" (ex: tester deux affectations simultanées, tester l'affectation d'un choriste à un poste de batteur - doit être possible mais avec avertissement ou bloqué selon la règle).

Contraintes de réalisation :

- **Performance** : Utilisation de `joinedload` pour récupérer l'activité et les affectations en une seule fois.
- **Atomicité** : Utilisation de transactions pour s'assurer qu'une affectation n'est pas créée si la validation échoue.

Format : Tableaux JIRA et sections Confluence claires.

 Analyse du besoin (Pour toi)

L'Épique 04 est **Critique** car elle lie tous tes modèles. Voici ce que tu vas obtenir et pourquoi c'est important :

1. **L'Intégrité Temporelle** : L'IA va devoir définir comment une **Activite** (qui a une date) impacte la `t_affectation`.
2. **Le Workflow de validation** : Contrairement aux épiques précédentes, ici on ne fait pas que du stockage, on fait de la **vérification**.
3. **La vue croisée** : C'est ici que tu pourras sortir des endpoints comme "Donne-moi le planning du Dimanche prochain avec tous les chanteurs et musiciens confirmés".

Priorité	Ticket	Pourquoi ?
----------	--------	------------

P0	Moteur de Collision Temporelle	Sans cela, le planning n'a aucune valeur de vérité.
P0	CRUD Activité & Slots	La structure de base indispensable.
P1	Service d'Affectation avec Validation	Lier les membres aux slots en vérifiant les rôles.
P1	Workflow de Statut (Draft/Published)	Empêcher les membres de voir un planning en cours de modification.
P2	Système de Notifications	Alerter automatiquement par email/push lors de la publication.

Priorité	Titre du Ticket	User Story & Détails Techniques	Critères d'Acceptation (AC)	
P0	[CORE] Moteur de Collision & Chevauchement	<i>En tant que Système, je veux détecter les conflits temporels afin d'empêcher les doubles affectations.</i>	1. Requête SQL optimisée avec index sur les dates. 2. Détection des conflits sur <code>t_indisponib</code>	

		Tech : Créer une fonction utilitaire <code>check_overlap(start, end, member_id)</code> .	<code>ilite</code> ET <code>t_affectation</code> . 3. Retourne une erreur 409 avec le détail du conflit.	
P0	[CRUD] Gestion Activités & Slots	<i>En tant que Planificateur, je veux structurer l'événement et ses créneaux.</i> Tech : CRUD pour <code>t_activite</code> et <code>t_slot</code> .	1. Création cascade : Activité → Planning -> Slots en une fois. 2. Utilisation de <code>joined load</code> pour la lecture. 3. Soft delete géré sur <code>delete_d_at</code> .	
P1	[SERVICE] Affectation avec		<i>En tant que Planificateur, je veux affecter un</i>	1. Vérification de l'existence du rôle

	Validation de Rôles		<i>membre en m'assurant de sa compétence.</i> Tech : Service AssignmentService. Vérification dans t_membre_role .	pour le membre. 2. Blocage ou Warning si rôle manquant (selon paramètre) . 3. Transaction atomique (commit global).
P1	[LOGIC] Workflow de Publication (Statut)	<i>En tant que Responsable, je veux passer le planning de BROUILLON à PUBLIÉ.</i> Tech : State Machine sur PlanningService.statut_code .	1. Les affectations ne sont visibles par le membre que si statut = PUBLIE . 2. Verrouillage des modifications si statut =	

			CLOTUR E .	
P2	[NOTIF] Déclencheur de Notification s de Service	<i>En tant que Membre, je veux être informé dès que mon planning est publié.</i> Tech : Hook post- publication.	1. Détection du passage au statut PUBLIE . 2. Envoi asynchron e (Celery/Ta sk) des notification s aux membres impactés.	

2. Backlog JIRA (Tickets Techniques)

Épique EP-04 : Moteur de Planning

Ticket ID	Titre & User Story	Critères d'Acceptation (AC)	Stratégie de Test (Pytest)
MLA-401	Orchestration Activité & Planning En tant que Planificateur, je veux créer une instance de	• POST /plannin gs : Crée un Planning Service lié à une Activite .	• Tester l'atomicité : Si la création de l'activité échoue, le planning ne doit pas être créé.

	planning pour une activité.	Statut par défaut : BROUILLON.	
MLA-402	Moteur d'Affectation avec Validation En tant que Planificateur, je veux affecter un chantre à un poste spécifique.	- POST /affectations : Reçoit chantre_id , voix_code ou instrument_code . - Retourne 409 Conflict avec détail si indisponible ou déjà pris.	Stress Test : Simuler 2 requêtes simultanées sur le même chantre (Race condition) via des verrous de base de données.
MLA-403	Publication et Verrouillage En tant que Responsable, je veux publier le planning pour informer les équipes.	- PATCH /plannings/{id}/publier . - Vérifie que chaque poste "Principal" est pourvu.	Tester qu'un utilisateur sans rôle "Chef de Chœur" reçoit une 403 Forbidden .

		• Envoi (mock) de notifications.	
MLA-404	Vue Croisée Planning (Read Model) En tant que Chantre, je veux voir mon planning personnalisé.	• GET <code>/me/plannings</code> : Retourne la liste des affectations avec les détails de l'activité. • Utilisation de joinedload pour la performance.	• Vérifier que le JSON de sortie contient bien le nom de l'instrument ou de la voix.

Contraintes de Réalisation & Architecture

Optimisation des Performances (Lead Dev Note)

Pour éviter l'effondrement de la base lors de la consultation des plannings mensuels, nous utiliserons le chargement lié :

1 Python

```

1 # Exemple de requête optimisée pour le ticket MLA-404
2 statement = (
3     select(PlanningService)
4     .options(
5         joinedload(PlanningService.activite),
6         joinedload(PlanningService.affectations).joinedload(Affectation.chantre)
7     )
8 )
9 
```

Atomicité & Transactions

Toute affectation doit être wrappée dans un bloc `try/except` avec un `session.begin_nested()` (savepoint) pour garantir que les validations de rôles techniques (Voix/Instrument) ne corrompent pas l'état du planning en cas d'erreur.

1. 📁 Structure des Répertoires

```
1 Plaintext
2
3 app/
4   ├── core/
5   │   └── database.py      # Configuration DB
6   ├── models/             # Vos SQLAlchemy Models (t_activite, etc.)
7   ├── repositories/       # Couche d'accès aux données (SQL)
8   │   ├── planning_repo.py
9   │   └── activite_repo.py
10  ├── services/            # Logique Métier (Validations, Algorithmes)
11  │   ├── planning_service.py
12  │   └── validation_service.py
13  ├── routes/              # API Endpoints (FastAPI)
14  └── planning_router.py
```

2. 📝 Couche Repository (`repositories/`)

Le repository ne contient que des requêtes SQL (via SQLAlchemy). Il ne prend aucune décision métier.

`repositories/planning_repo.py`

Fonction	Description	Requête SQL / SQLAlchemy
<code>get_planning_full</code>	Récupère le planning avec toute l'arborescence (optimisé N+1).	<code>select(PlanningService).options(selectinload(Slot.affectations)...))</code>
<code>get_membre_affectations</code>	Récupère toutes les affectations d'un membre sur une période.	<code>select(Affectation).where(Affectation.membre_id == id & date_overlap)</code>

<code>create_assignment</code>	Enregistre une affectation dans la base.	<code>session.add(affectation)</code>
--------------------------------	--	---------------------------------------

`repositories/activite_repo.py`

Fonction	Description	Requête SQL / SQLAlchemy
<code>create_activite</code>	CRUD Activité.	<code>session.add(activite)</code>
<code>get_activite</code>	Récupère une activité.	<code>session.get(Activite, id)</code>

3. 🧠 Couche Service (`services/`)

C'est ici que réside **l'intelligence du Moteur**.

`services/validation_service.py` (Moteur de Règles)

Fonction	Description	Logique
<code>check_member_availability</code>	Vérifie <code>t_indisponibilite</code> .	Retourne <code>False</code> si overlap.
<code>check_member_role</code>	Vérifie <code>t_membre_role</code> .	Retourne <code>False</code> si compétence manquante.
<code>check_double_booking</code>	Vérifie <code>t_affectation</code> .	Analyse les slots surchargés.

`services/planning_service.py` (Orchestrateur)

Fonction	Description	Logique
----------	-------------	---------

<code>assign_member</code>	Fonction Critique. Ordonne les validations et commit.	<code>begin_transaction()</code> → Validations → <code>commit()</code> ou <code>rollback()</code> .
<code>publish_planning</code>	Change le statut et déclenche les notifications.	Update <code>StatutPlanning</code> + Async Task.

4. 🌐 Couche Routes (`routes/`)

Le router valide les entrées (Pydantic) et appelle le service.

`routes/planning_router.py`

Route HTTP	Fonction	Description
<code>POST /activities</code>	<code>create_activity</code>	Crée la structure.
<code>GET /planning/{id}</code>	<code>get_planning</code>	Affiche le planning complet.
<code>POST /slots/{id}/assign</code>	<code>assign_member</code>	Tente une affectation (appel service).
<code>PATCH /planning/{id}</code>	<code>update_planning_status</code>	Publie ou clôture.

💡 Exemple de code complet pour le flux d'affectation

Voici comment le service orchestre la logique pour garantir la cohérence :

```

1 # services/planning_service.py
2
3 class PlanningService:
4     @staticmethod
5     def assign_member(session: Session, slot_id: str, member_id: str,
6 role_code: str):
7         # 1. Start Transaction
8         with session.begin():
9             # 2. Récupérer les données avec eager loading
10             slot = PlanningRepo.get_slot_with_details(session,
11 slot_id)
12
13             # 3. VALIDER RÈGLES MÉTIER
14             # Check indisponibilité
15             if not
16 ValidationService.check_member_availability(session, member_id,
17 slot.debut, slot.fin):
18                 raise ConflictException("Le membre est indisponible")
19
20             # Check double affectation
21             if not ValidationService.check_double_booking(session,
22 member_id, slot.debut, slot.fin):
23                 raise ConflictException("Le membre est déjà affecté
24 ailleurs")
25
26             # Check compétence
27             if not ValidationService.check_member_role(session,
28 member_id, role_code):
29                 # On peut choisir de bloquer ou de lever un
30 avertissement
31                 raise ValidationException("Compétence non valide")
32
33             # 4. CRÉER AFFECTATION
34             new_assign = Affectation(slot_id=slot_id,
35 membre_id=member_id, role_code=role_code)
36             PlanningRepo.create_assignment(session, new_assign)
37
38             return new_assign
39
40

```